

UNIT II

Introduction to Programming in C

Introduction to C

C is a programming language developed at AT & T's Bell Laboratories of USA in 1970. It was designed and written by a man named Dennis Ritchie. In the late seventies C began to replace the more familiar languages of that time like PL/I, ALGOL etc. No one pushed C. it wasn't made the official Bell Labs language. Thus, without any advertisement C's reputation spread and its pool of users grew. Ritchie seems to have been rather surprised that so many programmers preferred C to older languages like FORTRAN or PL/I or the newer ones like Pascal and APL.

Possibly why C seems so popular is because it is reliable, simple and easy to use. Out of the dozens of languages available, the prize of purity is often given to PASCAL.

Historical Development of C Language

C is a general-purpose language which has been closely associated with the UNIX operating system for which it was developed - since the system and most of the programs that run it are written in C. COBOL was being used for commercial applications, FORTRAN for Engineering and Scientific Applications and so on. At this stage people started thinking that instead of learning and using so many languages, each for a different purpose, why not use only one language. Which can program all possible applications. Therefore an international committee came out with a language called ALGOL 60. However, ALGOL 60 never really became popular because it seemed too abstract, too general. To reduce this abstractness and generality, a new language called combined Programming Language (CPL) was developed at Cambridge University. CPL was an attempt to bring ALGOL 60 down to earth. However, CPL turned out to be so big, having so many features, that it was hard to learn and difficult to implement.

Basic Combined Programming Language (BCPL), developed by Martin Richards at Cambridge University aimed to solve this problem by bringing CPL down to its basic good features. But unfortunately it turned out to be too less powerful and too specific. Around same time a language called B was written by Ken Thompson at AT & T's Bell Labs, as a further simplification of CPL. But like BCPL, B too and BCPL, added some of his own and developed C. Ritchie's main achievement is the restoration of the lost generality in BCPL and B, and still keeping it powerful.

ALGORITHM:

The word "algorithm" relates to the name of the mathematician Al-khowarizmi, which means a procedure or a technique. Software Engineer commonly uses an algorithm for planning and solving the problems. An algorithm is a sequence of steps to solve a particular problem or algorithm is an ordered set of unambiguous steps that produces a result and terminates in a finite time. Algorithm has the following characteristics

- **Input:** An algorithm may or may not require input
- **Output:** Each algorithm is expected to produce at least one result

..

- **Definiteness:** Each instruction must be clear and unambiguous.
- **Finiteness:** If the instructions of an algorithm are executed, the algorithm should terminate after finite number of steps

Advantages of algorithm

- It is a step-wise representation of a solution to a given problem, which makes it easy to understand.
- An algorithm uses a definite procedure.
- It is not dependent on any programming language, so it is easy to understand for anyone even without programming knowledge.
- Every step in an algorithm has its own logical sequence so it is easy to debug.

HOW TO WRITE ALGORITHMS

Step 1 **Define your algorithms input:** Many algorithms take in data to be processed, e.g. to calculate the area of rectangle input may be the rectangle height and rectangle width.

Step 2 **Define the variables:** Algorithm's variables allow you to use it for more than one place. We can define two variables for rectangle height and rectangle width as HEIGHT and WIDTH (or H & W). We should use meaningful variable name e.g. instead of using H & W use HEIGHT and WIDTH as variable name.

Step 3 **Outline the algorithm's operations:** Use input variable for computation purpose, e.g. to find area of rectangle multiply the HEIGHT and WIDTH variable and store the value in new variable (say) AREA. An algorithm's operations can take the form of multiple steps and even branch, depending on the value of the input variables.

Step 4 **Output the results of your algorithm's operations:** In case of area of rectangle output will be the value stored in variable AREA. if the input variables described a rectangle with a HEIGHT of 2 and a WIDTH of 3, the algorithm would output the value of 6.

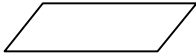
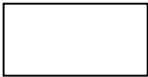
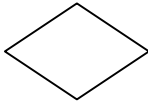
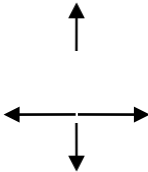
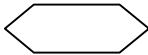
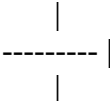
FLOWCHART:

Flowchart is the diagrammatic or pictorial representation of the algorithm for solving the problem. This is a tool used in the design of program. Flowchart consist of pictorial symbols. Flowchart helps a programmer to plan the procedure for the solution of a problem.

The first design of flowchart goes back to 1945 which was designed by John Von Neumann. Unlike an algorithm, Flowchart uses different symbols to design a solution to a problem. It is another commonly used programming tool. By looking at a Flowchart one can understand the operations and sequence of operations performed in a system. Flowchart is often considered as a blueprint of a design used for solving a specific problem.

Advantages of flowchart:

- Flowchart is an excellent way of communicating the logic of a program.
- Easy and efficient to analyze problem using flowchart.
- During program development cycle, the flowchart plays the role of a blueprint, which makes program development process easier.
- After successful development of a program, it needs continuous timely maintenance during the course of its operation. The flowchart makes program or system maintenance easier.
- It is easy to convert the flowchart into any programming language code.

Symbol Name	Symbol	function
Oval		Used to represent start and end of flowchart
Parallelogram		Used for input and output operation
Rectangle		Processing: Used for arithmetic operations and data-manipulations
Diamond		Decision making. Used to represent the operation in which there are two/three alternatives, true and false etc
Arrows		Flow line Used to indicate the flow of logic by connecting symbols
Circle		Page Connector
		Off Page Connector
		Predefined Process /Function Used to represent a group of statements performing one processing task.
		Preprocessor
		Comments

..

Algorithm & Flowchart to find the sum of two numbers

Algorithm

Step-1 Start

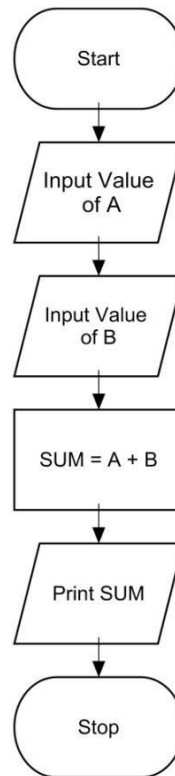
Step-2 Input first numbers say A

Step-3 Input second number say B

Step-4 $SUM = A + B$

Step-5 Display

SUM Step-6 Stop



..

Algorithm & Flowchart to convert temperature from Celsius to Fahrenheit

C : temperature in Celsius
F : temperature Fahrenheit

Algorithm

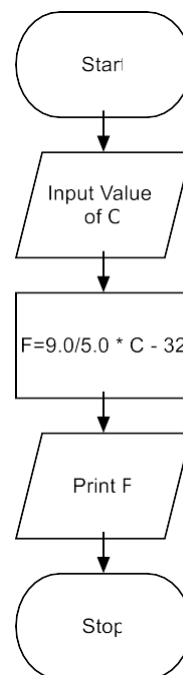
Step-1 Start

Step-2 Input temperature in Celsius say C

Step-3 $F = (9.0/5.0 \times C) + 32$

Step-4 Display Temperature in Fahrenheit F

Step-5 Stop



Algorithm & Flowchart to convert temperature from Fahrenheit to Celsius

C : temperature in Celsius
F : temperature Fahrenheit

Algorithm

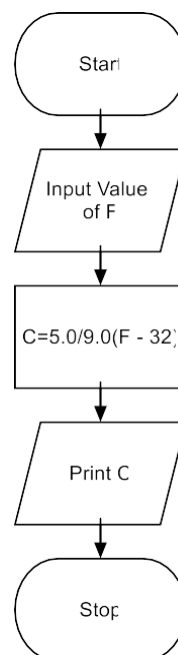
Step-1 Start

Step-2 Input temperature in Fahrenheit say F

Step-3 $C = 5.0/9.0 (F - 32)$

Step-4 Display Temperature in Celsius C

Step-5 Stop



Algorithm & Flowchart to find Area and Perimeter of Square

L : Side Length of Square

AREA : Area of Square

PERIMETER : Perimeter of Square

Algorithm

Step-1 Start

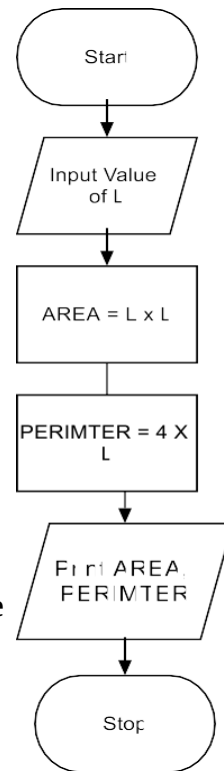
Step-2 Input Side Length of Square say L

Step-3 $\text{Area} = L \times L$

Step-4 $\text{PERIMETER} = 4 \times L$

Step-5 Display AREA,

PERIMETER Step-6 Stop



Algorithm & Flowchart to find Area and Perimeter of Rectangle

L : Length of Rectangle

B : Breadth of Rectangle

AREA : Area of Rectangle

PERIMETER : Perimeter of Rectangle

Algorithm

Step-1 Start

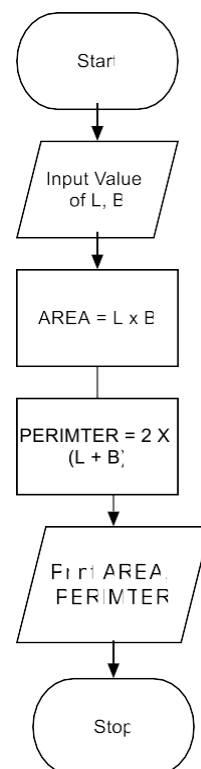
Step-2 Input Side Length & Breadth say L,

B Step-3 $\text{Area} = L \times B$

Step-4 $\text{PERIMETER} = 2 \times (L + B)$

Step-5 Display AREA,

PERIMETER Step-6 Stop



..

Algorithm & Flowchart to find Area and Perimeter of Circle

R : Radius of Circle

AREA : Area of

Circle

PERIMETER : Perimeter of Circle

Algorithm

Step-1 Start

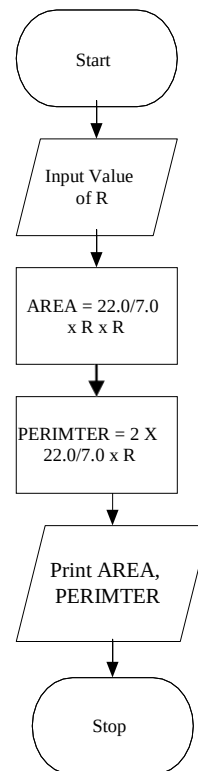
Step-2 Input Radius of Circle say

R Step-3 $\text{Area} = 22.0/7.0 \times R \times R$

Step-4 $\text{PERIMETER} = 2 \times 22.0/7.0 \times R$

Step-5 Display AREA,

PERIMETER Step-6 Stop



Algorithm & Flowchart to find Area & Perimeter of Triangle (when three sides are given)

A : First Side of Triangle

B : Second Side of Triangle

C : Third Side of Triangle

AREA : Area of Triangle

PERIMETER : Perimeter of Triangle

Algorithm

Step-1 Start

Step-2 Input Sides of Triangle

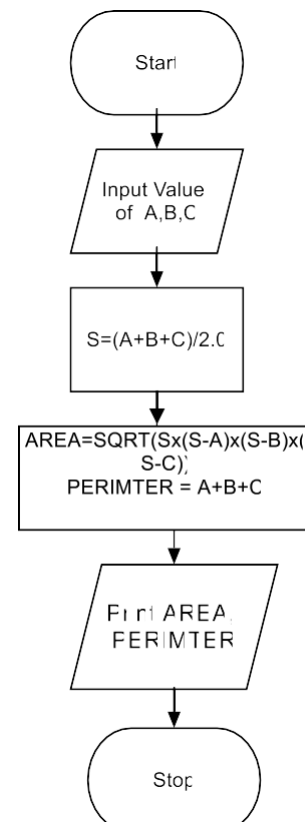
A,B,C Step-3 $S = (A + B + C) / 2.0$

Step-4 $\text{AREA} = \text{SQRT}(S \times (S-A) \times (S-B) \times (S-C))$

Step-5 $\text{PERIMETER} = S1 + S2 + S3$

Step-6 Display AREA,

PERIMETER Step-7 Stop



..

Algorithm & Flowchart to find Simple Interest

P : Principle Amount
N : Time in Years
R : % Annual Rate of Interest
SI : Simple Interest

Algorithm

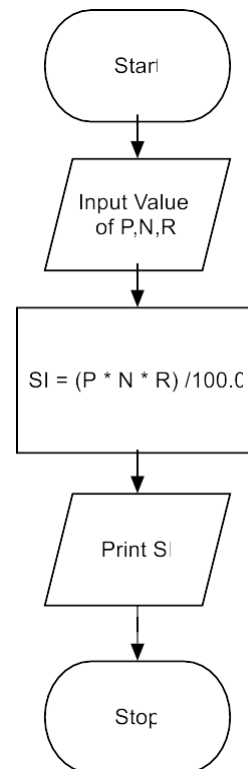
Step-1 Start

Step-2 Input value of P, N, R

Step-3 $SI = (P \times N \times R) / 100.0$

Step-4 Display SI

Step-6 Stop



Algorithm & Flowchart to find Compound Interest

P : Principle Amount
N : Time in Years
R : % Annual Rate of Interest
CI : Compound Interest

Algorithm

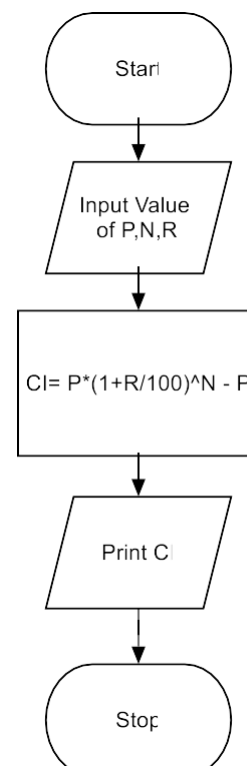
Step-1 Start

Step-2 Input value of P, N, R

Step-3 $CI = P(1+R/100)^N - P$

Step-4 Display CI

Step-6 Stop



Algorithm & Flowchart to Swap Two Numbers using Temporary Variable

Algorithm

Step-1 Start

Step-2 Input Two Numbers Say NUM1, NUM2

Step-3 Display Before Swap Values NUM1,

NUM2 Step-4 TEMP = NUM1

Step-5 NUM1 =

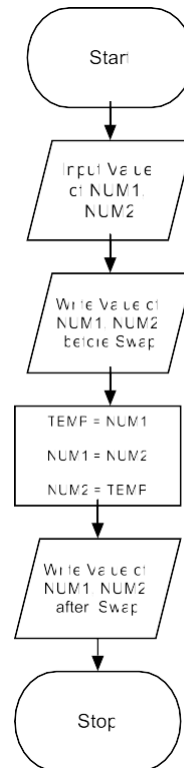
NUM2 Step-6

NUM2 =

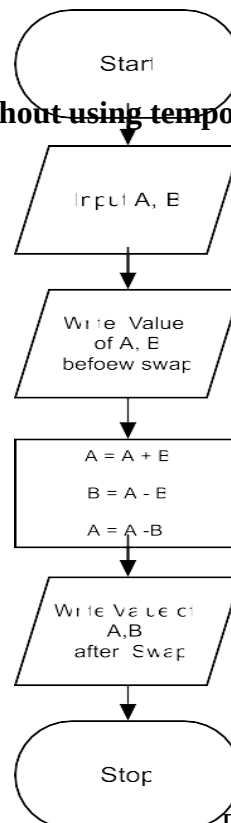
NUM1

Step-7 Display After Swap Values

NUM1, NUM Step-8 Stop



Algorithm & Flowchart to Swap Two Numbers without using temporary variable



Algorithm

Step-1 Start

Step-2 Input Two Numbers Say A,B

Step-3 Display Before Swap Values A,

B Step-4 $A = A + B$

Step-5 $B = A -$

B Step-6 $A = A -$

B

Step-7 Display After Swap Values A, B

Step-8 Stop

Flowchart are not related to any programming language. Hence they can be implemented in any programming language.

Flowchart are easy to understand. The data flow of the program can be easily understood by flowcharts and errors in it can be easily recovered.

When we need to incorporate some more facilities within program, then it is easy to incorporate them through flowchart.

APPLICATIONS OF C LANGUAGE

C Programming is a best-known programming language. C Programming is near to machine as well as human so it is called as Middle-level Programming Language. C Programming can be used to do a variety of tasks such as networking related, OS related.

1. C language is used for **creating computer applications**
2. Used in writing Embedded software
3. Firmware for various electronics, industrial and communications products which use micro-controllers.
4. It is also used in developing verification software, test code, simulators etc. for various applications and hardware products.
5. For **Creating Compiler** of different Languages which can take input from other language and convert it into lower level machine dependent language.
6. C is used to implement different Operating System Operations. UNIX kernel is completely developed in C Language.

ADVANTAGES OF C LANGUAGE

- It contains a powerful data definition. The data type supported are characters, alphanumeric, integers, long integer, float, double. It also supports string manipulation in the form of character array.
- C supports a powerful set of operators.
- It also supports powerful graphics programming and directly operators with hardware. Execution of program is faster.
- An assembly code is also inserted into C programs.
- C programs such as compilers, Operating systems can be developed in C. For example the popular operating system UNIX is developed in C.
- The C language has 32 keywords and about 145 library functions and near about 30 header files.
- C works closely with machines and matches assembly language in many ways

Structure of C Programming

- 1) Documentation Section
- 2) Header Section [link section]
- 3) Definition Section
- 4) Global Declaration
- 5) Main Section
- 6) Sub Program Section

1) Documentation Section

We can write title of program in that section using comments line there are two types of comment line

- a) **Single line comment**:-it is starting with // [double slash] this type of comment line are used for single line comment.
- b) **Multi line Comment** /* */:-This type of comment line is used to give additional information about the program in two or more then two lines this type of comment line is started with /* & end with */ there is no effect on the program by using comment line.

2) Header Section[link section]

In this section we can use the Header file such as # 'stdio.h' & 'conio.h' , string.h, such as Header file are used to give c linking support to the functions used in c program.

Stdio.h stands for slandered I/O file. Which support to standard I/O function as printf() & scanf().

Similarly conio.h is stand for console I/O hidder file which support to console I/O functions such as clrscr(); & getch().

3) Definition Section

In definition section we define constant value by using #defines ex:-#define pie 3.14 In above example to define constant value 3.142 variable.

4) Global Declaration Section

There are some variables that are used in more than one function such variables are called global variable they are declared in the global declaration section that in outside of all function.

5) Main() Section

Every C program have only one main() function is start with ({) & close with (}) main function have main two part i.e declaration part & execution part.

a) Declaration part:-

In this part we declare the variables before using program. In declaration part we declare variables with their particular data types such as int, float, char etc.

b) Execution Part:-

In this part of main() section we use execution statements in main() section every statement & function are terminated by (;).

6) Sub program section:-

In sub program section we can use user defined function. User defined function are function which are defined by user.

Simple C Program

```
/* This is my first 'c' program */
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    printf("Wel come C programming");
    getch();
}
```

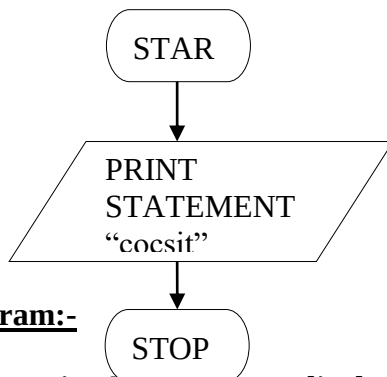
```
//Write a program two addition of two numbers
#include<stdio.h>
#include<conio.h>
Void main()
{
Clrscr()
Printf("The addition of two number is %d",20+30);
Getch();
}
```

//Write A Program to display the name of college.

Algorithm:-

- 1.Start
- 2.Print statement "cocsit"
- 3.stop

Flowchart:-



Program:-

/* Write A Program to display the name of college.*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    printf("*****out/put*****");
    printf("Hello cocsit");

    getch();
}
```

*****out/put*****
Hello cocsit

C Tokens

The Character set

Every language has its own character set ex;- In English language there are 26 alphabets are used to create word, sentence, paragraph.

Similarly 'C' language has its own character sets. The character set of 'C' language is nothing but combination of alphabets, digits & special symbols 'C' inserts the uppercase letter A—Z lower case letter a---z the digits 0----9 & special character. The alphabets & digits together are called as alphanumeric character.

These characters are used as building blocks from basic program element.

Following table shows the valid alphabets, digits & special symbol allowed in 'C'

Alphabets

Uppercase -

A	B	C	-----	Z.
---	---	---	-------	----

Lowercase -

a	b	C	-----	Z
---	---	---	-------	---

Digits:-

0	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Special Characters

,	Comma
;	Semi column
:	Column
?	Question Mark
“	Double quotation
	Vertical Bar
\	Back Slash
/	Slash
~	Tile
_	Under score
\$	Dollar sign
#	Hash
%	Percentage
&	Ampere Sign
*	Astric

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

-	Minus
+	Plus
<	Less sign/ opening angle
>	Grater sign/ closing angle
(	Left parentheses
)	Right parentheses
[	Left Bracket
]	Right Bracket
{	Left Brace
}	Right Brace

White spacious

The 'C' compiler ignores white space unless they are a part of string constant white space may be used to separate words, white space include blank space, horizontal tab.

Horizontal tab - /t
New line - /n

By using this character set we can create program, function name, variable name. 'C' uses certain combination of characters such as, /b,/n,/t. to represent special condition such as back space, new line & horizontal tab these characters combination are known as escape sequence.

IDENTIFIERS AND KEYWORDS

Identifiers

- Identifiers are names that are given to various program elements such as variables, function & array name. there are certain rules we should be follow while running & identifiers are
- Identifier consists of letters & digits in any order except the first character should be letter
- Both uppercase & lowercase letters are permitted although lowercase letter are commonly used.
- Uppercase & lowercase letters are not interchangeable [an uppercase letters is not equivalent to corresponding lowercase letter
- The underscore [_] character can be included & it is considered as a letter.
- An identifiers name up to 8 characters many compilers allow an identifiers name up to 31 characters.

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

- Following names are valid identifiers x, xyz, sum, nau, basic_sal.
First character must be letters.

The following names are valid identifiers

X	y12	sum_1	_tempera
Names	area	text_rate	TABLE

The following names are not valid identifiers

7h	The first character must be a letter
X"	Illegal characters (").
Rder-no	Illegal characters (-).
Rror flag	Illegal characters (blank space).

C Keywords

Keywords are the words whose meaning has already been explained to the C compiler. The keywords cannot be used as variable names because if we do so we are trying to assign a new meaning to the keyword, which is not allowed by the computer.

Auto	Break	Case	Char	Const	Continue
Default	do	Double	Else	Enum	extern
Float	far	For	Goto	If	Int
Long	near	Register	Return	Short	Signed
Static	struct	Switch	Typedef	Union	Unsigned
Void	while				

There are 32 keywords used in C all keywords must be written in lower case.

Some compilers may also include some or all of the following keywords.

Ada	Far	Near	Asm	Fortran	Pascal
Entry	Huge				

Variables

A variable is a data name that may be used to store the data value. These variables are given to location in the memory of computer where we are storing the data. There are some rules available for creation and declaration of variables.

```
#include<stdio.h>
#include<conio.h>
```



```
void main()
{
int a,b,c;
clrscr();
printf("Enter the number");
scanf("%d",&a,&b);
c=a+b
printf("The addition is %d",c);
getch();
}
```

Types of variables

In C, a quantity which may change during program execution is called a variable. Variable names are names given to locations in the memory of computer where different constants are stored. This location can contain integer, real or character constants. This is because a constant stored in a location with a particular type of variable name can hold only that type of constant. Ex:- a constant stored in a memory location with an integer variable name must be an integer constant. One stored in location with a real variable name must be a real constant and the one stored in location with a character variable name must be a character constant.

Rules for Constructing Variable Name.

- a) A variable name is any combination of 1 to 8 alphabets, digits or underscores. Some compilers allow variable names whose length could be up to 40 characters.
- b) The first character in the variable name must be an alphabet.
- c) No commas or blanks are allowed within a variable name.
- d) No special symbol other than an underscore (as in gross_sal) can be used in a variable name.

Ex:- int I,net_sal;

Float a;

Char code;-

Data type

C language is rich in data type like other languages 'C' language has also its own data

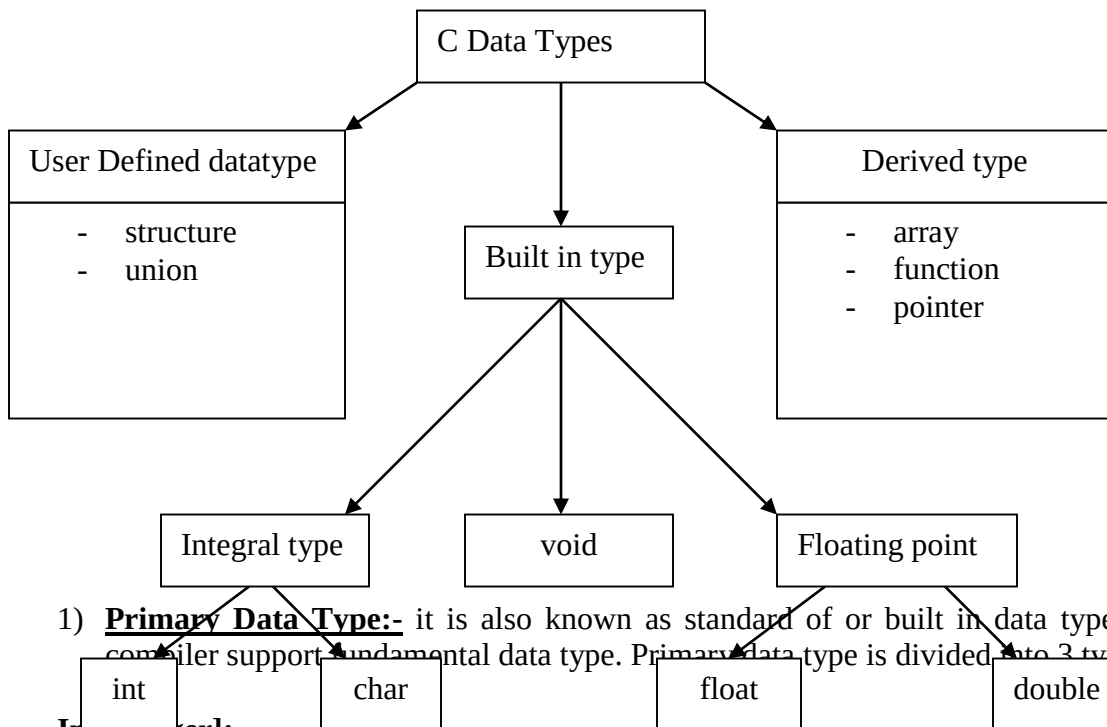
types data types are used to define or declare the type [data type] of particular variable, constant, function etc.

In C language there are three classes of data types.

- 1) Primary data type.
- 2) Derived data type.
- 3) User-defined data types.

Data type are always written in small letters each data types has it's own feature depending upon a condition the particular data types is used in the name of data type we cannot use in valid space.

Data Type	Range	Size in Bite	Size in Byte	Format String
Char	-128 to 127	8	1	%c
Unsigned char	0 to 255	8	1	%c
Int	-32768 to 32767	16	2	%i or %d
Unsigned int	0 to 65535	16	2	%u
long int	-2147483648 to 2147483647	32	4	%ld
Unsigned long int	0 to 4294967295	32	4	%lu
Float	3.4e-38 to 3.4e+38	32	4	%f or %g
Double	1.7e-308 to 1.7e+308	64	8	%lf
Long double	3.4e-4932 to 1.1e+4932	80	10	%lf



1) **Primary Data Type:-** it is also known as standard or built in data type all compiler support fundamental data type. Primary data type is divided into 3 type.

Int Integer:-

Integer are defined by keyword 'int' integer are store for no we cannot store decimal point no in that data type integer occupies one word of storage i.e 2 bytes or 16 bits. The range of integer data type is -32768 to 32767 'C' has support to classes of integer storage namely int shor int & long int variable than they increases the range of value

as well as size of it. 32 bit word length can store an integer ranging from - 2,147,483,648 to 2147483647.

Ex:- Int a=5; int b=6;

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c;
    a=10;
    b=20;
    c=a+b;
    printf("\n A addition of two number is %d",c);
    getch();
}
```

Float

The real no are known as floating value. These no contain fractions part. This is defined by keyword 'float' float value requires 4 bytes or 32 bits. The range of float is 3.4×10^{-38} to 3.4×10^{38} .

Ex:- float pie=3.14;

```
#include<stdio.h>
#include<conio.h>
{
    float a,b,c;
    clrscr();
    a=3.2;
    b=1.5;
    c=a+b;
    printf("\n Addition of two number %f",c);
    getch();
}
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int sub1=40,sub2=50,sub3=60,sum;
    float avg;
    clrscr();
    sum=sub1+sub2+sub3;
```

```
printf("\n Addition of three subject is %d",sum);  
avg=sum/3;  
printf("\n average of three subject is %f",avg);  
getch();  
}
```

character (char)

Generally those data types are used when we have to storage a single character or string to variable. It requires 1 byte or 8 beats to store in memory. It is defined by keyword as 'char' the range of char is -128 to 127.

Ex:-char name;
Name='a';
Char name[15]="Bhosle";

Double

It is very similar to float data type the major difference between float & double data type is size & range of the double data type is always greater than float data type size of double data type is 64 beats, 8 bytes range of double data type is $1.7e-308$ to $1.7e+308$.

Void Type

The Void type has no value. This is usually used to specify the type of functions. The type of a function is said to be void when it does not return any value to the calling function. It can also play the role of a generic type, meaning that it can represent any of the other standard types.

User defined data type

These data type are defined by using keyword 'typedef' it takes following format.

Syntax :- typedef type identifier name

Ex:- typedef int mark

The example will demonstrate the use of user defined data type.

#include<stdio.h>

```
#include<conio.h>
typedef int mark
Void main()
{
    mark sub1,sub2,sub3,sum,avg;
    printf("\nEnter the mark of three subject");
    scanf("%d%d%d",&sub1,&sub2,&sub3);
    sum=sub1+sub2+sub3;
    avg=sum/3;
    printf("\nThe total of three subject is%d",sum);
    printf("\nThe average of three subject is %d",avg);
    getch();
}
```

here marks indicate integer data type but sub1, sub2,sub3 indirectly marks referees to standard data type int.

Derived data type

Some time these derived data type are also called user defined data type which is derived from fundamental data type. The derived data type are

ex:- array, pointer, function.

Escape Sequences

These are the special characters starting with ‘\’ which makes the computer to line out of the normal sequence.

Computer has some default sequences e.g computer always prints data in a line unless we ask the computer to create a new line etc. the following are the escape sequences.

Constant	Meaning
\n	New line
\t	Horizontal tab
\v	Vertical tab
\a	Alert (bell)
\b	Back space
\f	Form feed

Constant	Meaning
\r	Carriage return
\0	Null
\'	Single quote
\"	Double quote
\?	Question mark
\\	Back slash

//Write a program to swap two value by using third variable

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a,b,c;
```

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

```
clrscr();
printf("\nEnter the First number");
scanf("%d",&a);
printf("\nEnter the Second Number");
scanf("%d",&b);
c=a;
a=b;
b=c;
printf("\n After swap A=%d",a);
printf("\n After swap B=%d",b);
getch();
}
```

*****out/put*****

```
Enter the First number 23
Enter the Second Number 55
After swap A= 55
After swap B= 23
```

//Write a program to swap two value by using third variable

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a,b;
clrscr();
printf("\nEnter the First number');
scanf("%d",&a);
printf("\nEnter the Second Number");
scanf("%d",&b);
a=a+b;
b=a-b;
a=a-b;
printf("\n After swap A=%d",a);
printf("\n After swap B=%d",b);
getch();
}
```

*****out/put*****

Enter the First number 23
Enter the Second Number 55
After swap A= 55
After swap B= 23

/*write a program to reveres the given number*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
int no,rev,a1,a2,a3;
clrscr();
printf("Enter the number");
scanf("%d",&no);
a1=no%10;
no=no/10;
a2=no%10;
no=no/10;
a3=no;
rev=100*a1+10*a2+a3;
printf("Revers number is %d",rev);
getch();
}
```

*****out/put*****

Enter the number 123
Entered number is 321

Operators :-

An operators is a symbol which tells the computer to perform mathematic or logical operation C supports following 8 types of operators

- 1) Arithmetic operators
- 2) Relational operators
- 3) Logical operators
- 4) Assignment operators
- 5) Increment & Decrement operators
- 6) Conditional operators
- 7) Bit-wise operators
- 8) Special operators

1) Arithmetic operators

C Language supports all arithmetic operators this operators provides same function as in other languages this operators can operate on any built in data type in C following list gives the arithmetic operators with meaning.

+, -, *, %

Integer division ignores frication parts in the result c doesn't support an operators for exponential

A+B

A and b is operands

//Addition of two number

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a;
    float b,c;
    clrscr();
    printf("\nEnter the two number");
    scanf("%d%f",&a,&b);
    c=a+b;
    printf("Addition of two number is %f",c);
    getch();
}
```


2) Relational operators

If we want to compare two values then Relational operators are used C supports six Relational operators

- 1 >
- 2 >=
- 3 <
- 4 <=
- 5 ==
- 6 !=

The simple format of Relational operators as follows

Expression Relational operator's expression

The value of real expression is dissention statement if

//Write a program to use various relational operators and display their return values.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    clrscr();
    printf("\nCondition : Return Values \n");
    printf("\n10!=10      : %5d",10!=10);
    printf("\n10==10      : %5d",10==10);
    printf("\n10>=10       : %5d",10>=10);
    printf("\n10<=10       : %5d",10<=10);
    printf("\n10!=9        : %5d",10!=9);
    getch();
}
```

3) logical operators

C language support three logical operators

&&	Logical And
	Logical or
!	Logical Not

a<b && x==30

a>b || b>c

Logical 'AND' (&&)		
Condition1	Condition 2	Result
True	False	False
False	True	False
True	True	True
False	False	False

Logical 'OR' (&&)		
Condition1	Condition 2	Result
True	False	True
False	True	True
True	True	True
False	False	False

Logical 'NOT' (!)	
True	False
False	True

4) Assignment Operators

The assignment operators are used to assign values or result of the expression to variables C support for the assignment operators that is

=

ex:-a=10;

5) Increment and Decrement operators

Prefix Increment and Decrement operators

In the prefix operators the values of variables is Increment and Decrement First and then assigned to the expression

++a --a

a=3

y=++a

y=4

postfix operators

a post fix operators first assign the values to variables on left side and then increment the operand

a=5;

y=a++;

y=6

6) Conditional operators

Conditional operators

Expression ? expression2 : expression3

? :

a=3;

b=4;

x=(a<b)? a:b;

//write a program to find out the given number greater or not by using conditional operators.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
Void main()
```

```
{
```

```
    Int a,b;
```

```
    Printf("Enter two number");
```

```
    Scanf("%d%d",&a,&b);
```

```
    a>b?printf("A is greater number"):Printf("B is greater number");
```

```
    getch();
```

```
}
```

in above expression a is less than b if this condition is true then value of a will be assigned to the expression and if given condition is false then value of b will be assigned to the expression

7) Bit-wise operators

These operators are used for manipulation of data at bit level we can use these operators for testing the bits, shift right and left we can't apply the bit wise operators with float and double following list gives the operators and their meaning

This operator moves bit patterns either to the left or to the right. The shift operators are represented by Left shift << and Right shift >>.

<< Left

10

0	0	0	0	1	0	1	0
---	---	---	---	---	---	---	---

0	0	1	0	1	0	0	0
>>Right							
10							
0	0	0	0	1	0	1	0
		0	0	0	0	1	0

8) Special operators

Comma operators this operator is used to link or connect the related expression

Values(a=3,b=4,a+b)

X=7

3 is assign to variables a 4 will be assign to b after that x=7 addition of two numbers

// Write a program to use of comma (,) operator

```
#include<stdio.h>
#include<conio.h>
void main()
{
clrscr();
printf(“ Subtraction = %d\n Addition =%d”,5-3,5+6);
getch();
}
```

Input And Output Statement

Introduction

Reading the data from the input devices and displaying the results on the screen, are the two main tasks of any program. To perform these tasks user friendly C has a number of input and output functions. When a program needs data, it takes the data through the input functions and sends results obtained through the output functions. The input/output functions are the link between the user and the VDU.

There are number of I/O functions in C based on the data types. The input/output functions are classified in to two types

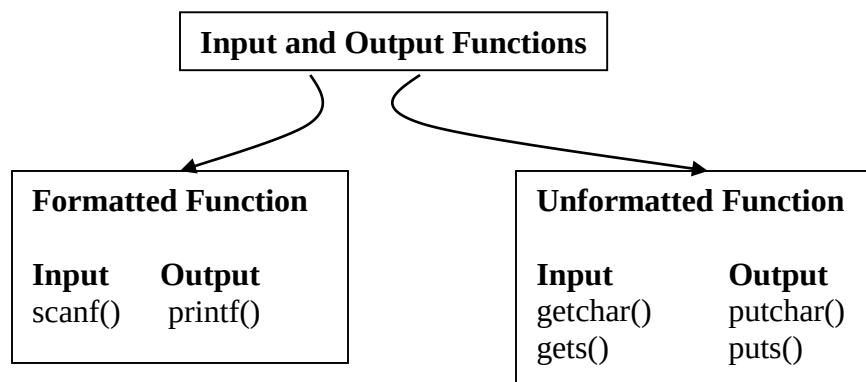
- 4) Formatted functions
- 5) Unformatted function.

1) Formatted Function:-

The formatted input/output functions read and write all types of data values. They require conversion symbol to identify the data type. They can be used for both reading and writing of all data values. The formatted functions return the values after execution. The return value is equal to the number of variables successfully read/ written.

2) Unformatted function:-

The formatted input/output functions only work with the character data type. There is no need to convert the data. In case values of other data types are passed to these functions, they are treated as the character data. The unformatted function also return values, but the return value of unformatted function is always the same.



Formatted Function:-

printf():-

This function is used to display result on the screen it can be used to display any combination of numerical value as well as char or string. It requires conversion symbol and variable names to print the data. The conversion symbol and variable names should be same in number.

Syntax:-

```
printf("<Format string>" ,arg1,arg2,...);
```

Ex:-

```
void main()
{
    int a=3;
    float b=5;
    char c='b';
    printf("%d    %f    %c",a,b,c);
}
```

//display the ascii value

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int y=65;
    clrscr()
    printf("%c    %d",y,y);
    getch();
}
```

scanf():-

The input data or information can be enter to the computer through a standard input device by using scanf() function. This function can be used to enter any combination of numerical value, single char, or strings this function return the number of data item that have been successful we can declare the scanf() function as follow.

```
scanf("<Format string>" ,&arg1,&arg2,...);
```

In the format string various format specifies are used

Ex:- scanf("%d",&a);

The argument1,argument2,.....are the argument at that represent that indusial data item in the format string these argument are return of variables. The scanf() statement requires '&' operator called address operator. The address operators print the memory

location of the variable. Here in the scanf() statement the role of '&' operator is to indicate the memory location of the variable. So that value read would be placed at that location.

Ex:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c;
    clrscr();
    scanf("%d%d",&a,&b);
    c=a+b;
    printf("%d",c);
    getch();
}
```

Getchar() Function:-

Single character can be entered into the computer using 'c' library function getchar(); the get character function is a part of standard I/O function. It written a single character from a standard input device [keyboard] the function does not require any argument a pair of empty parentheses must follow the word getchar(). The getchar take following form

Variable name=getchar();

Variable name is a valid C name that has been declared as char type. When this statement is encountered, the computer waits until a key is pressed and then assign this character as a value to getchar function. Since getchar is used on the right-hand side of an assignment statement,

Syntax:-

Char name;
Name=getchar();

Ex:-

```
Void main()
{
    char c;
    clrscr();
    c=getchar();
    putchar(c);
    getch();
}
```

Putchar() Function:-

This function prints one character on the screen at a time which is read by the standard input device such as keyboard.

Syntax:-

`putchar(name);`

Ex:-

```
Void main()
{
    char c;
    clrscr();
    c=getchar();
    putchar(c);
    getch();
}
```

puts() function:-

This function is used to display the string or collection of character to the output device such as monitor.

Syntax:-

`Char name[20];
puts(name);`

Ex:-

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char name[20];
    clrscr();
    printf("\nEnter the name");
    gets(name);
    printf("My name is %s",name);
    getch();
}
```


gets() function:-

This function is used to accepting any string or collection of character from input device such as keyboard.

Syntax:-

```
Char name[20];  
Name=gets();
```

Ex:-

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    char name[20];  
    clrscr();  
    printf("\nEnter the name");  
    gets(name);  
    printf("\n My name is ");  
    puts(name);  
    getch();  
}
```

Conversion Specifiers used in c

Data type	Conversion Symbol
Integer	%d or %i
Unsigned	%u
Long signed	%ld
Long unsigned	%ld
Unsigned hexadecimal	%x
Unsigned octal	%o
Float	%f or %g
Double	%lf
Char	%c
Unsigned char	%c
String	%s